



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Capturas Correspondientes de la Guía Ape 05
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	1ro Software - Paralelo "B"
Alumnos participantes:	Loor Velez Jhon Alejandro Bravo López Jordan Samuel Cajiao Valdivieso Paulo Alessandro Balarezo Pérez Diego Sebastián Pomaquero Chango Katherine Soledad
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Ejercicio 1

2.1.1 Código C++

2.1.1.1 Clases y Métodos

```
class Estudiante {
private:
    string cedula;
    string nombre;
    string carrera;
    double promedio;

public:
    // Constructor
    Estudiante(string _cedula, string _nombre, string _carrera, double _promedio) {
        cedula = _cedula;
        nombre = _nombre;
        carrera = _carrera;
        promedio = _promedio;
    }

    // Métodos para obtener los atributos (Getters)
    string getCedula() { return cedula; }
    string getNombre() { return nombre; }
    string getCarrera() { return carrera; }
    double getPromedio() { return promedio; }
};
```

2.1.1.2 Código Principal o Fuente

```
int main() {
    string ced, nom, carr;
    double prom;

    cout << "=== REGISTRO DE ESTUDIANTES (C++) ===" << endl;

    cout << "Ingrese la cedula: ";
    cin >> ced;
    cin.ignore(); // Limpiar el buffer para usar getline

    cout << "Ingrese el nombre completo: ";
    getline(cin, nom);

    cout << "Ingrese la carrera: ";
    getline(cin, carr);

    cout << "Ingrese el promedio: ";
    cin >> prom;

    // Creación del objeto
    Estudiante estudiante(ced, nom, carr, prom);
}
```



```
// Escritura de archivos (Modo ios::app para agregar datos sin borrar los anteriores)
ofstream archivoOut;
archivoOut.open("estudiantes.txt", ios::app);

if (archivoOut.is_open()) {
    // Escribimos los datos separados por comas
    archivoOut << estudiante.getCedula() << ","
    << estudiante.getNombre() << ","
    << estudiante.getCarrera() << ","
    << estudiante.getPromedio() << "\n";

    archivoOut.close(); // Buena práctica: cerrar siempre el archivo
    cout << "\n[CONFIRMACION]: Estudiante registrado con exito en estudiantes.txt." << endl;
} else {
    cout << "\n[ERROR]: No se pudo abrir el archivo para escribir." << endl;
}

return 0;
}
```

2.1.1.3 Ejecución del Programa

```
==== REGISTRO DE ESTUDIANTES ====
Ingrese la cedula: 0850981929
Ingrese el nombre completo: Samuel Bravo
Ingrese la carrera: Software
Ingrese el promedio: 9

[CONFIRMACION]: Estudiante registrado con exito en estudiantes.txt.

Process returned 0 (0x0)   execution time : 122.127 s
Press any key to continue.
```

2.1.2 Código Java

2.1.2.1 Clases y Métodos



```
// Definición de la Clase Estudiante
class Estudiante {
    private String cedula;
    private String nombre;
    private String carrera;
    private double promedio;

    // Constructor
    public Estudiante(String cedula, String nombre, String carrera, double promedio) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.promedio = promedio;
    }

    // Métodos de acceso (Getters)
    public String getCedula() { return cedula; }
    public String getNombre() { return nombre; }
    public String getCarrera() { return carrera; }
    public double getPromedio() { return promedio; }
}
```

2.1.2.2 Código Principal o Fuente

```
// Clase Principal
public class Main {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.println("=== REGISTRO DE ESTUDIANTES (JAVA) ===");

        System.out.print("Ingrese la cédula: ");
        String cedula = teclado.nextLine();

        System.out.print("Ingrese el nombre completo: ");
        String nombre = teclado.nextLine();

        System.out.print("Ingrese la carrera: ");
        String carrera = teclado.nextLine();

        System.out.print("Ingrese el promedio: ");
        double promedio = teclado.nextDouble();

        // Instancia del Objeto
        Estudiante estudiante = new Estudiante(cedula, nombre, carrera, promedio);
```

```
/*
Escritura de archivos:
El segundo parámetro 'true' en FileWriter habilita el modo Append (adición)
*/
try (FileWriter fw = new FileWriter("estudiantes.txt", true);
    PrintWriter pw = new PrintWriter(fw)) {

    // Escritura formateada
    pw.println(estudiante.getCedula() + "," +
        estudiante.getNombre() + "," +
        estudiante.getCarrera() + "," +
        estudiante.getPromedio());

    System.out.println("\n[CONFIRMACIÓN]: Estudiante registrado con éxito en estudiantes.txt.");

} catch (IOException e) {
    System.out.println("\n[ERROR]: Ocurrió un problema al escribir en el archivo: " + e.getMessage());
}
}
```



2.1.2.3 Ejecución del Programa

Output Clear

```
=== REGISTRO DE ESTUDIANTES (JAVA) ===  
Ingrese la c?dula: 0850981929  
Ingrese el nombre completo: Samuel Bravo  
Ingrese la carrera: Sotfware  
Ingrese el promedio: 9  
  
[CONFIRMACI?N]: Estudiante registrado con ?xito en estudiantes.txt.  
  
=== Code Execution Successful ===
```

2.2 Ejercicio 2

2.2.1 Código C++

2.2.1.1 Clases y Métodos

```
Start here x apees.h x apees.cpp x  
11 class Estudiante {  
12 private:  
13     string cedula;  
14     string nombre;  
15     string carrera;  
16     double promedio;  
17  
18 public:  
19     // Constructor por defecto (necesario para inicializar arreglos en C++)  
20     Estudiante() {  
21         cedula = "";  
22         nombre = "";  
23         carrera = "";  
24         promedio = 0.0;  
25     }  
26  
27     // Constructor parametrizado  
28     Estudiante(string _cedula, string _nombre, string _carrera, double _promedio) {  
29         cedula = _cedula;  
30         nombre = _nombre;  
31         carrera = _carrera;  
32         promedio = _promedio;  
33     }  
34  
35     // Métodos Getters para acceder a los atributos encapsulados  
36     string getCedula() { return cedula; }  
37     string getNombre() { return nombre; }  
38     string getCarrera() { return carrera; }  
39     double getPromedio() { return promedio; }
```

2.2.1.2 Código Principal o Fuente



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Start here X apee.h X apee.cpp X
44 // =====
45 int main() {
46 // Requerimiento: Uso de Arreglos (Capacidad para 100 estudiantes)
47 Estudiante listaEstudiantes[100];
48 int contador = 0; // Cuenta los estudiantes existentes
49
50 // Requerimiento: Lectura de archivos
51 ifstream archivoIn("estudiantes.txt");
52
53 if (!archivoIn.is_open()) {
54     cout << "[ERROR]: No se pudo abrir el archivo estudiantes.txt o el archivo no existe." << endl;
55     return 1;
56 }
57
58 string linea;
59 // Requerimiento: Ciclo para leer el archivo línea por línea
60 while (getline(archivoIn, linea)) {
61     if (linea.empty()) continue; // Ignorar líneas vacías de seguridad
62
63     stringstream ss(linea);
64     string ced, nom, carr, tempProm;
65     double prom;
66
67     // Separar la línea usando la coma como delimitador (Tokenización)
68     getline(ss, ced, ',');
69     getline(ss, nom, ',');
70     getline(ss, carr, ',');
71     getline(ss, tempProm, ',');
72
73     // Convertir el promedio que viene como texto a tipo double
74     prom = stod(tempProm);
75
76     // Guardar el objeto instanciado dentro del arreglo
77     listaEstudiantes[contador] = Estudiante(ced, nom, carr, prom);
78     contador++;
79 }
80
81 // Cerrar el archivo después de cargar los datos en memoria
82 archivoIn.close();
83
84 // =====
85 // MOSTRAR RESULTADOS Y CÁLCULOS
86 // =====
87 cout << "=== MOSTRANDO TODOS LOS REGISTROS ALMACENADOS ===" << endl;
88 double sumaPromedios = 0;
89
90 // Ciclo para recorrer el arreglo de objetos
91 for (int i = 0; i < contador; i++) {
92     cout << "Estudiante #" << (i + 1) << endl;
93     cout << "  -> Cedula: " << listaEstudiantes[i].getCedula() << endl;
94     cout << "  -> Nombre: " << listaEstudiantes[i].getNombre() << endl;
95     cout << "  -> Carrera: " << listaEstudiantes[i].getCarrera() << endl;
96     cout << "  -> Promedio: " << listaEstudiantes[i].getPromedio() << endl;
97     cout << "-----" << endl;
98 }
99
100 // Acumular los promedios para la estadística general
101 sumaPromedios += listaEstudiantes[i].getPromedio();
102 }
103
104 // Calcular el promedio general controlando la división para cero
105 double promedioGeneral = (contador > 0) ? (sumaPromedios / contador) : 0.0;
106
107 // Mostrar requerimientos de salida finales
108 cout << "\n>>> REPORTE METRICO FINAL <<<" << endl;
109 cout << "Total de estudiantes existentes: " << contador << endl;
110 cout << "Promedio general del grupo: " << promedioGeneral << endl;
111
112 return 0;
113 }
```

2.2.1.3 Ejecución del Programa



```
"C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\apeee.exe"

=== MOSTRANDO TODOS LOS REGISTROS ALMACENADOS ===
Estudiante #1
-> Cedula: 0850981929
-> Nombre: Samuel Bravo
-> Carrera: Software
-> Promedio: 9
-----

>>> REPORTE METRICO FINAL <<<
Total de estudiantes existentes: 1
Promedio general del grupo: 9

Process returned 0 (0x0)   execution time : 2.551 s
Press any key to continue.
```

2.2.2 Código Java

2.2.2.1 Clases y Métodos

```
class Estudiante {
    private String cedula;
    private String nombre;
    private String carrera;
    private double promedio;

    // Constructor parametrizado
    public Estudiante(String cedula, String nombre, String carrera,
        double promedio) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.promedio = promedio;
    }

    // Métodos Getters (Encapsulamiento)
    public String getCedula() { return cedula; }
    public String getNombre() { return nombre; }
    public String getCarrera() { return carrera; }
    public double getPromedio() { return promedio; }
}
```

2.2.2.2 Código Principal o Fuente



```
public class Main {
    public static void main(String[] args) {
        // Requerimiento: Uso de Arreglos (Inicialización de arreglo
        // estático con tope de 100)
        Estudiante[] listaEstudiantes = new Estudiante[100];
        int contador = 0; // Cuenta cuántos estudiantes existen en el
        // archivo

        // Requerimiento: Lectura de archivos mediante bloques
        // seguros
        try (FileReader fr = new FileReader("estudiantes.txt");
            BufferedReader br = new BufferedReader(fr)) {

            String linea;
            // Requerimiento: Ciclo para leer el archivo hasta que no
            // haya más registros
            while ((linea = br.readLine()) != null) {
                // Evitar procesar líneas en blanco accidentales
                if (linea.trim().isEmpty()) continue;

                // Separar la cadena por comas
                String[] campos = linea.split(",");

                String ced = campos[0];
                String nom = campos[1];
                String carr = campos[2];
                double prom = Double.parseDouble(campos[3]); //
                // Conversión de Texto a Double

                // Almacenar el objeto instanciado dentro de la
                // posición actual del arreglo
                listaEstudiantes[contador] = new Estudiante(ced, nom,
                    carr, prom);
                contador++;
            }

        } catch (IOException e) {
            System.out.println("[ERROR]: No se pudo leer el archivo o
            aún no ha sido creado: " + e.getMessage());
            return;
        }
    }
}
```



```
} catch (IOException e) {  
    System.out.println("[ERROR]: No se pudo leer el archivo o  
        aún no ha sido creado: " + e.getMessage());  
    return;  
}  
  
// =====  
// PROCESAMIENTO DE DATOS Y MÉTRICAS  
// =====  
System.out.println("=== MOSTRANDO TODOS LOS REGISTROS  
    ALMACENADOS ===");  
double sumaPromedios = 0;  
  
// Ciclo para recorrer el arreglo de objetos estructurado  
for (int i = 0; i < contador; i++) {  
    System.out.println("Estudiante #" + (i + 1));  
    System.out.println(" -> Cédula:  " + listaEstudiantes[i]  
        .getCedula());  
    System.out.println(" -> Nombre:  " + listaEstudiantes[i]  
        .getNombre());  
    System.out.println(" -> Carrera: " + listaEstudiantes[i]
```




```
System.out.println(" -> Promedio: " + listaEstudiantes[i]
    .getPromedio());
System.out.println("
    -----");

// Acumular la suma de notas para el cálculo final
sumaPromedios += listaEstudiantes[i].getPromedio();
}

// Requerimiento: Mostrar promedio general controlando
// división para cero
double promedioGeneral = (contador > 0) ? (sumaPromedios /
    contador) : 0.0;

// Salidas en pantalla solicitadas por el enunciado
System.out.println("\n>>> REPORTE ESTADÍSTICO FINAL <<<");
System.out.println("Total de estudiantes existentes: " +
    contador);
System.out.println("Promedio académico general:      " +
    promedioGeneral);
}
```

2.2.2.3 Ejecución del Programa

```
=== MOSTRANDO TODOS LOS REGISTROS ALMACENADOS ===
Estudiante #1
-> Cedula: 0850981929
-> Nombre: Samuel Bravo
-> Carrera: Software
-> Promedio: 9
-----

>>> REPORTE METRICO FINAL <<<
Total de estudiantes existentes: 1
Promedio general del grupo:      9
```

2.3 Ejercicio 3

2.3.1 Código C++

2.3.1.1 Clases y Métodos



C Producto.h X	Main.cpp	≡ inventario.txt
----------------	----------	------------------

Actividades > output > C Producto.h > ...

```
1  // Librerías necesarias
2  #include <iostream>
3  using namespace std;
4
5  // Creación de la clase Producto
6  class Producto {
7
8      // Atributos privados
9      private:
10         int codigo;
11         string nombre;
12         int cantidad;
13         double precio;
14
15     public:
16
17         // Constructor vacío
18         Producto() {
19
20             codigo = 0;
21             nombre = "";
22             cantidad = 0;
23             precio = 0;
24         }
25
26         // Métodos SET para guardar datos
27         void setCodigo(int c) {
28             |   codigo = c;
29         }
30
31         void setNombre(string n) {
32             |   nombre = n;
```



```
33     }
34
35     void setCantidad(int cant) {
36         cantidad = cant;
37     }
38
39     void setPrecio(double p) {
40         precio = p;
41     }
42
43     // Métodos GET para obtener información
44     int getCodigo() {
45         return codigo;
46     }
47
48     string getNombre() {
49         return nombre;
50     }
51
52     int getCantidad() {
53         return cantidad;
54     }
55
56     double getPrecio() {
57         return precio;
58     }
59
60     // Método para calcular el total del producto
61     double calcularTotal() {
62
63         return cantidad * precio;
64     }
65
66     // Método para mostrar información
67     void mostrarProducto() {
68
69         cout << "\nCodigo: " << codigo << endl;
70         cout << "Nombre: " << nombre << endl;
71         cout << "Cantidad: " << cantidad << endl;
72         cout << "Precio: $" << precio << endl;
73         cout << "Total: $" << calcularTotal() << endl;
74     }
75 };
```

2.3.1.2 Código Principal o Fuente



```
Product.h  Main.cpp  inventario.txt
Actividades > output > Main.cpp > main()
1  // Librerías necesarias
2  #include <iostream>
3  #include <fstream>
4  #include "Producto.h"
5
6  using namespace std;
7
8  int main() {
9
10     // Variable para la cantidad de productos
11     int cantidadProductos;
12
13     // Variables temporales
14     int codigo;
15     int cantidad;
16     string nombre;
17     double precio;
18
19     // Variable para el total general
20     double totalInventario = 0;
21
22     // Pedir cuántos productos desea registrar el usuario
23     cout << "Ingrese la cantidad de productos: ";
24     cin >> cantidadProductos;
25
26     // Crear arreglo de objetos
27     Producto productos[cantidadProductos];
28
29     // Crear archivo para guardar datos
30     ofstream archivo("inventario.txt");
31
32     // Ciclo para registrar productos
```



```
33     for(int i = 0; i < cantidadProductos; i++) {
34
35         cout << "\n===== " << endl;
36         cout << "REGISTRO DEL PRODUCTO " << i + 1 << endl;
37         cout << "===== " << endl;
38
39         // Ingreso de datos
40         cout << "Ingrese el codigo: ";
41         cin >> codigo;
42
43         cin.ignore();
44
45         cout << "Ingrese el nombre: ";
46         getline(cin, nombre);
47
48         cout << "Ingrese la cantidad: ";
49         cin >> cantidad;
50
51         cout << "Ingrese el precio: ";
52         cin >> precio;
53
54         // Guardar datos en el objeto
55         productos[i].setCodigo(codigo);
56         productos[i].setNombre(nombre);
57         productos[i].setCantidad(cantidad);
58         productos[i].setPrecio(precio);
59
60         // Calcular total general
61         totalInventario += productos[i].calcularTotal();
62
63         // Guardar información en el archivo
64         archivo << "Codigo: " << codigo << endl;
65
66         archivo << "Nombre: " << nombre << endl;
67         archivo << "Cantidad: " << cantidad << endl;
68         archivo << "Precio: $" << precio << endl;
69         archivo << "Total: $" << productos[i].calcularTotal() << endl;
70         archivo << "-----" << endl;
71     }
72
73     // Cerrar archivo
74     archivo.close();
75
76     // Mostrar productos registrados
77     cout << "\n===== INVENTARIO ===== " << endl;
78
79     for(int i = 0; i < cantidadProductos; i++) {
80         productos[i].mostrarProducto();
81     }
82
83     // Mostrar total general del inventario
84     cout << "\nValor total del inventario: $" << totalInventario << endl;
85
86     return 0;
87 }
```

2.3.1.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
C Producto.h  Main.cpp X  inventario.txt
Actividades > output > Main.cpp > main()
b2
PROBLEMAS 6  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS C:\Users\Usuario\OneDrive\Ape05> cd 'c:\Users\Usuario\OneDrive\Ape05\Actividades\output\output'
PS C:\Users\Usuario\OneDrive\Ape05\Actividades\output\output> & .\'Main.exe'
Ingrese la cantidad de productos: 3

=====
REGISTRO DEL PRODUCTO 1
=====
Ingrese el codigo: 30
Ingrese el nombre: Cafe
Ingrese la cantidad: 2
Ingrese el precio: 5

=====
REGISTRO DEL PRODUCTO 2
=====
Ingrese el codigo: 31
Ingrese el nombre: Pollo
Ingrese la cantidad: 1
Ingrese el precio: 10

=====
REGISTRO DEL PRODUCTO 3
=====
Ingrese el codigo: 32
Ingrese el nombre: Galletas
Ingrese la cantidad: 8
Ingrese el precio: 2

Ingrese el precio: 2

===== INVENTARIO =====

Codigo: 30
Nombre: Cafe
Cantidad: 2
Precio: $5
Total: $10

Codigo: 31
Nombre: Pollo
Cantidad: 1
Precio: $10
Total: $10

Codigo: 32
Nombre: Galletas
Cantidad: 8
Precio: $2
Total: $16

Valor total del inventario: $36
PS C:\Users\Usuario\OneDrive\Ape05\Actividades\output\output>
```



```
Producto.h Main.cpp inventario.txt X
Actividades > output > inventario.txt
1  Código: 30
2  Nombre: Cafe
3  Cantidad: 2
4  Precio: $5
5  Total: $10
6  -----
7  Código: 31
8  Nombre: Pollo
9  Cantidad: 1
10 Precio: $10
11 Total: $10
12 -----
13 Código: 32
14 Nombre: Galletas
15 Cantidad: 8
16 Precio: $2
17 Total: $16
18 -----
19
```

2.3.2 Código Java

2.3.2.1 Clases y Métodos



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
J Producto.java X J Main.java E inventariojava.txt
J Producto.java > Producto
1 // Clase Producto
2 public class Producto {
3
4     // Atributos privados
5     private int codigo;
6     private String nombre;
7     private int cantidad;
8     private double precio;
9
10    // Constructor vacío
11    public Producto() {
12
13        codigo = 0;
14        nombre = "";
15        cantidad = 0;
16        precio = 0;
17    }
18
19    // Métodos SET
20    public void setCodigo(int codigo) {
21        this.codigo = codigo;
22    }
23
24    public void setNombre(String nombre) {
25        this.nombre = nombre;
26    }
27
28    public void setCantidad(int cantidad) {
29        this.cantidad = cantidad;
30    }
31
32    public void setPrecio(double precio) {
```

```
33        this.precio = precio;
34    }
35
36    // Métodos GET
37    public int getCodigo() {
38        return codigo;
39    }
40
41    public String getNombre() {
42        return nombre;
43    }
44
45    public int getCantidad() {
46        return cantidad;
47    }
48
49    public double getPrecio() {
50        return precio;
51    }
52
53    // Método para calcular el total
54    public double calcularTotal() {
55
56        return cantidad * precio;
57    }
58
59    // Método para mostrar información
60    public void mostrarProducto() {
61
62        System.out.println("\nCodigo: " + codigo);
```




```
63      System.out.println("Nombre: " + nombre);
64      System.out.println("Cantidad: " + cantidad);
65      System.out.println("Precio: $" + precio);
66      System.out.println("Total: $" + calcularTotal());
67      }
68  }
```

2.3.2.2 Código Principal o Fuente



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
J Producto.java J Main.java X inventariojava.txt
J Main.java > Main > main(String[])
1 // Librerías necesarias
2 import java.util.Scanner;
3 import java.io.FileWriter;
4 import java.io.IOException;
5
6 public class Main {
7
8     Run | Debug
9     public static void main(String[] args) {
10
11         // Objeto Scanner para ingresar datos
12         Scanner entrada = new Scanner(System.in);
13
14         // Pedir cantidad de productos
15         System.out.print(s: "Ingrese la cantidad de productos: ");
16         int cantidadProductos = entrada.nextInt();
17
18         // Crear arreglo de objetos
19         Producto productos[] = new Producto[cantidadProductos];
20
21         // Variables temporales
22         int codigo;
23         int cantidad;
24         String nombre;
25         double precio;
26
27         // Variable para guardar el total general
28         double totalInventario = 0;
29
30         try {
31             // Crear archivo
32             FileWriter archivo = new FileWriter(fileName: "inventariojava.txt");
33
34             // Ciclo para registrar productos
35             for(int i = 0; i < cantidadProductos; i++) {
36
37                 // Crear objeto
38                 productos[i] = new Producto();
39
40                 System.out.println(x: "\n=====");
41                 System.out.println("REGISTRO DEL PRODUCTO " + (i + 1));
42                 System.out.println(x: "=====");
43
44                 // Ingreso de datos
45                 System.out.print(s: "Ingrese el codigo: ");
46                 codigo = entrada.nextInt();
47
48                 entrada.nextLine();
49
50                 System.out.print(s: "Ingrese el nombre: ");
51                 nombre = entrada.nextLine();
52
53                 System.out.print(s: "Ingrese la cantidad: ");
54                 cantidad = entrada.nextInt();
55
56                 System.out.print(s: "Ingrese el precio: ");
57                 precio = entrada.nextDouble();
58
59                 // Guardar datos
60                 productos[i].setCodigo(codigo);
61                 productos[i].setNombre(nombre);
```



```
61     productos[i].setNombre(nombre);
62     productos[i].setCantidad(cantidad);
63     productos[i].setPrecio(precio);
64
65     // Calcular total general
66     totalInventario += productos[i].calcularTotal();
67
68     // Guardar en archivo
69     archivo.write("Codigo: " + codigo + "\n");
70     archivo.write("Nombre: " + nombre + "\n");
71     archivo.write("Cantidad: " + cantidad + "\n");
72     archivo.write("Precio: $" + precio + "\n");
73     archivo.write("Total: $" + productos[i].calcularTotal() + "\n");
74     archivo.write(str: "-----\n");
75 }
76
77 // Cerrar archivo
78 archivo.close();
79
80 } catch(IOException e) {
81
82     System.out.println(x: "Error al crear el archivo");
83 }
84
85 // Mostrar inventario
86 System.out.println(x: "\n===== INVENTARIO =====");
87
88 for(int i = 0; i < cantidadProductos; i++) {
89
90     productos[i].mostrarProducto();
91 }
92
93 // Mostrar total general
94 System.out.println("\nValor total del inventario: $" + totalInventario);
95
96 // Cerrar Scanner
97 entrada.close();
98 }
99 }
```

2.3.2.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
J Producto.java J Main.java X inventariojava.txt
J Main.java > Main
6 public class Main {
PROBLEMAS 6 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\Usuario\OneDrive\Ape05> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDetailsIn
eptionMessages' '-cp' 'C:\Users\Usuario\AppData\Roaming\Code\User\workspaceStorage\31acb19243734b59fbb452f
06bf91\redhat.java\jdt_ws\Ape05_d35a3370\bin' 'Main'
Ingrese la cantidad de productos: 2

=====
REGISTRO DEL PRODUCTO 1
=====
Ingrese el codigo: 12
Ingrese el nombre: Azucar
Ingrese la cantidad: 3
Ingrese el precio: 5

=====
REGISTRO DEL PRODUCTO 2
=====
Ingrese el codigo: 13
Ingrese el nombre: Cocoa
Ingrese la cantidad: 1
Ingrese el precio: 8

===== INVENTARIO =====
```

```
===== INVENTARIO =====

Codigo: 12
Nombre: Azucar
Cantidad: 3
Precio: $5.0
Total: $15.0

Codigo: 13
Nombre: Cocoa
Cantidad: 1
Precio: $8.0
Total: $8.0

Valor total del inventario: $23.0
PS C:\Users\Usuario\OneDrive\Ape05> |
```

```
J Producto.java J Main.java X inventariojava.txt X
X inventariojava.txt
1 Codigo: 12
2 Nombre: Azucar
3 Cantidad: 3
4 Precio: $5.0
5 Total: $15.0
6 -----
7 Codigo: 13
8 Nombre: Cocoa
9 Cantidad: 1
10 Precio: $8.0
11 Total: $8.0
12 -----
13
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



2.4 Ejercicio 4

2.4.1 Código C++

2.4.1.1 Clases y Métodos



```
class Contacto {  
public:  
    string nombre, telefono, correo;  
  
    Contacto(string n, string t, string c) {  
        nombre = n;  
        telefono = t;  
        correo = c;  
    }  
  
    string toString() {  
        return nombre + "," + telefono + "," + correo;  
    }  
};
```

```
void agregar() {  
    ofstream file(archivo, ios::app);  
    string n, t, c;  
  
    cout << "Nombre: ";  
    getline(cin, n);  
    cout << "Telefono: ";  
    getline(cin, t);  
    cout << "Correo: ";  
    getline(cin, c);  
  
    Contacto con(n, t, c);  
    file << con.toString() << endl;  
  
    file.close();  
    cout << "Contacto guardado\n";  
}
```



```
void mostrar() {  
    ifstream file(archivo);  
    string linea;  
  
    while (getline(file, linea)) {  
        cout << linea << endl;  
    }  
  
    file.close();  
}
```

```
void buscar() {  
    ifstream file(archivo);  
    string linea, nombre;  
    bool encontrado = false;  
  
    cout << "Nombre a buscar: ";  
    getline(cin, nombre);  
  
    while (getline(file, linea)) {  
        if (linea.find(nombre) != string::npos) {  
            cout << "Encontrado: " << linea << endl;  
            encontrado = true;  
        }  
    }  
  
    if (!encontrado)  
        cout << "No encontrado\n";  
  
    file.close();  
}
```

2.4.1.2 Código Principal o Fuente



```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Contacto {
public:
    string nombre, telefono, correo;

    Contacto(string n, string t, string c) {
        nombre = n;
        telefono = t;
        correo = c;
    }

    string toString() {
        return nombre + "," + telefono + "," + correo;
    }
};

string archivo = "agenda.txt";

void agregar() {
    ofstream file(archivo, ios::app);
    string n, t, c;
```




```
    cout << "Nombre: ";
    getline(cin, n);
    cout << "Telefono: ";
    getline(cin, t);
    cout << "Correo: ";
    getline(cin, c);

    Contacto con(n, t, c);
    file << con.toString() << endl;

    file.close();
    cout << "Contacto guardado\n";
}

void mostrar() {
    ifstream file(archivo);
    string linea;

    while (getline(file, linea)) {
        cout << linea << endl;
    }

    file.close();
}
```



```
void buscar() {
    ifstream file(archivo);
    string linea, nombre;
    bool encontrado = false;

    cout << "Nombre a buscar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) != string::npos) {
            cout << "Encontrado: " << linea << endl;
            encontrado = true;
        }
    }

    if (!encontrado)
        cout << "No encontrado\n";

    file.close();
}

int main() {
    int op;
```



```
do {  
    cout << "\n1. Agregar\n2. Mostrar\n3. Buscar\n4. Salir\n";  
    cout << "Opcion: ";  
    cin >> op;  
    cin.ignore();  
  
    switch (op) {  
        case 1: agregar(); break;  
        case 2: mostrar(); break;  
        case 3: buscar(); break;  
    }  
  
} while (op != 4);  
  
return 0;  
}
```

2.4.1.3 Ejecución del Programa



```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 1
Nombre: Michael
Telefono: 0979350748
Correo: msanchez3405@gmail.com
Contacto guardado
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 2
Michael,0979350478,mruiz3405@gmail.com
Michael,0979350748,msanchez3405@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 3
Nombre a buscar: Michael
Encontrado: Michael,0979350478,mruiz3405@gmail.com
Encontrado: Michael,0979350748,msanchez3405@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Salir
Opcion: 4
PS C:\Users\HP\Documents\ALGORITMOS\Agenda> []
```

2.5 Ejercicio 5

2.5.1 Código C++

2.5.1.1 Clases y Métodos

```
class Libro {
private:
    string codigo;
    string titulo;
    string autor;
    string estado; // "Terminado" o "En proceso"

public:
    Libro(string c, string t, string a, string e) {
        codigo = c;
        titulo = t;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



2.5.1.2 Código Principal o Fuente



```
int main() {
    Biblioteca miBiblioteca;
    int opcion;

    do {
        cout << "\n--- MENU BIBLIOTECA VIRTUAL ---\n";
        cout << "1. Registrar Libro\n";
        cout << "2. Buscar Libro\n";
        cout << "3. Mostrar Disponibles y En Proceso\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;
        cin.ignore(); // Limpiar el buffer del salto de línea

        if (opcion == 1) {
            string cod, tit, aut, est;
            int estOpcion;
            cout << "Código: "; getline(cin, cod);
            cout << "Título: "; getline(cin, tit);
            cout << "Autor: "; getline(cin, aut);
            cout << "Estado (1. Terminado / 2. En proceso): "; cin >> estOpcion;
            est = (estOpcion == 1) ? "Terminado" : "En proceso";

            Libro nuevoLibro(cod, tit, aut, est);
            miBiblioteca.registrarLibro(nuevoLibro);
        } else if (opcion == 2) {
            string criterio;
            cout << "Ingrese código o título a buscar: ";
            getline(cin, criterio);
        }
    } while (opcion != 4);
}
```

2.5.1.3 Ejecución del Programa

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 1
Código: 2378
Título: POO Introduccion para Principiantes
Autor: Linus Torvalds
Estado (1. Terminado / 2. En proceso): 2
¡Libro registrado con éxito en el archivo!

--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 1
Código: 2589
Título: El Principito
Autor: Antonie de Saint Exupery
Estado (1. Terminado / 2. En proceso): 1
```



2.5.2 Código Java

2.5.2.1 Clases y Métodos

```
class Libro {
    private int    codigo;
    private String titulo;
    private String autor;
    private String estado; // "disponible" o "prestado"

    // Constructor
    public Libro(int codigo, String titulo, String autor, String estado) {
        this.codigo = codigo;
        this.titulo = titulo;
        this.autor  = autor;
        this.estado = estado;
    }

    // Getters
    public int    getCodigo() { return codigo; }
    public String getTitulo() { return titulo; }
    public String getAutor()  { return autor;  }
    public String getEstado() { return estado; }

    // Mostrar info del libro en consola
    public void mostrar() {
        System.out.printf(format: "%-6d %-30s %-22s %s\n",
                           codigo, titulo, autor, estado);
    }

    // Convertir a línea CSV para guardar en archivo
}
```

2.5.2.2 Código Principal o Fuente



```
public class LibrosVirtual {
    Run | Debug
    public static void main(String[] args) {
        Biblioteca bib = new Biblioteca();
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println(x: "\n");
            System.out.println(x: "    BIBLIOTECA VIRTUAL    ");
            System.out.println(x: "    ");
            System.out.println(x: "    1. Registrar libro    ");
            System.out.println(x: "    2. Buscar libro      ");
            System.out.println(x: "    3. Ver disponibles   ");
            System.out.println(x: "    4. Ver prestados     ");
            System.out.println(x: "    5. Ver todos los libros ");
            System.out.println(x: "    0. Salir              ");
            System.out.println(x: "    ");
            System.out.print(s: "Opcion: ");

            opcion = Integer.parseInt(sc.nextLine());

            switch (opcion) {
                case 1 -> bib.registrarLibro(sc);
                case 2 -> bib.buscarLibro(sc);
                case 3 -> bib.mostrarDisponibles();
                case 4 -> bib.mostrarPrestados();
                case 5 -> bib.mostrarTodos();
            }
        } while (opcion != 0);
    }
}
```

2.5.2.3 Ejecución del Programa

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 1
Código: 2378
Título: POO Introduccion para Principiantes
Autor: Linus Torvalds
Estado (1. Terminado / 2. En proceso): 2
¡Libro registrado con éxito en el archivo!

--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 1
Código: 2589
Título: El Principito
Autor: Antonie de Saint Exupery
Estado (1. Terminado / 2. En proceso): 1
¡Libro registrado con éxito en el archivo!
```




```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 2
Ingrese código o título a buscar: 2378

--- Libro Encontrado ---
Código: 2378 | Título: POO Introduccion para Principiantes | Autor: Linus Torvalds | Estado: En proceso
```

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 3

=== LIBROS TERMINADOS (Disponibles para lectura completa) ===
• Cod: 2589 - El Principito (Antonie de Saint Exupery)

=== LIBROS EN PROCESO (Lectura en curso/Prestados) ===
• Cod: 2378 - POO Introduccion para Principiantes (Linus Torvalds)
```

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Buscar Libro
3. Mostrar Disponibles y En Proceso
4. Salir
Seleccione una opcion: 4
PS C:\Users\disba\Documents\Diegui\UTA Primer Semestre\VSCodeProjects>
```

2.6 Ejercicio 6

2.6.1 Código C++

2.6.1.1 Clases y Métodos



```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>

using namespace std;

// =====
// FUNCIONES AUXILIARES DE VALIDACIÓN
// =====

// Valida de forma básica que la fecha tenga el formato numérico DD/MM/AAAA
bool validarFecha(const string& fecha) {
    if (fecha.length() != 10) return false;
    if (fecha[2] != '/' || fecha[5] != '/') return false;

    // Verifica que el resto sean números
    for (int i = 0; i < 10; i++) {
        if (i != 2 && i != 5) {
            if (fecha[i] < '0' || fecha[i] > '9') return false;
        }
    }
    return true;
}

// Solicita la fecha y fuerza al ciclo a repetirse hasta que sea válida
string solicitarFecha() {
    string fecha;
    while (true) {
        cout << " Ingrese fecha (DD/MM/AAAA): ";
        cin >> fecha;
        if (validarFecha(fecha)) {
            return fecha;
        }
        cout << " [Error] Formato de fecha incorrecto. Use DD/MM/AAAA.\n";
    }
}

// Fuerza al usuario a ingresar únicamente 'A' o 'F'
char solicitarEstado() {
    char estado;
    while (true) {
```

2.6.1.2 Código Principal o Fuente



```
int main() {
    int opcion;

    // Ciclo principal del menú de control de asistencia
    do {
        cout << "\n===== \n";
        cout << "    SISTEMA CONTROL DE ASISTENCIA \n";
        cout << "===== \n";
        cout << "1. Registrar Asistencia\n";
        cout << "2. Mostrar Historial y Faltas\n";
        cout << "3. Salir del programa\n";
        cout << "===== \n";
        cout << "Seleccione una opcion (1-3): ";

        // Validación para evitar bucles infinitos si ingresan letras en el menú
        if (!(cin >> opcion)) {
            cout << "Opcion no valida. Intente de nuevo.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (opcion) {
            case 1:
                registrarAsistencia();
                break;
            case 2:
                mostrarHistorialYFaltas();
                break;
            case 3:
                cout << "\nSaliendo del sistema de asistencia... ¡Hasta luego!\n";
                break;
            default:
                cout << "[Error] La opcion elegida no existe. Intente con 1, 2 o 3.\n";
        }
    } while (opcion != 3); // Rompe el ciclo cuando la opción es 3

    return 0;
}
```

2.6.1.3 Ejecución del Programa



```
=====
      SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
-----
Seleccione una opcion (1-3): 1

--- NUEVO REGISTRO DE ASISTENCIA ---
Nombre del estudiante: Paulo
Ingrese fecha (DD/MM/AAAA): 25/12/2004
Ingrese estado (A = Asistio, F = Falto): A

>> Asistencia registrada con exito en el sistema.

=====
      SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
-----
Seleccione una opcion (1-3): 2

=====
              HISTORIAL DE ASISTENCIA
=====
%-20s %-15s %-10s
-----
Paulo                25/12/2004        Asistio
=====
Total de registros procesados: 1
Total de FALTAS contabilizadas: 0
=====

=====
      SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
-----
Seleccione una opcion (1-3): |
```

2.6.2 Código Java

2.6.2.1 Clases y Métodos



```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

// =====
// CLASE PRINCIPAL: SistemaAsistencia
// =====
public class Ape5Ejercicio6 {

    private static final String NOMBRE_ARCHIVO = "asistencia.txt";
    private static final Scanner teclado = new Scanner(System.in);

    // Valida de forma básica que la fecha tenga el formato numérico DD/MM/AAAA
    public static boolean validarFecha(String fecha) {
        if (fecha.length() != 10) return false;
        if (fecha.charAt(index: 2) != '/' || fecha.charAt(index: 5) != '/') return false;

        // Verifica que los demás caracteres sean dígitos
        for (int i = 0; i < 10; i++) {
            if (i != 2 && i != 5) {
                if (fecha.charAt(i) < '0' || fecha.charAt(i) > '9') return false;
            }
        }
        return true;
    }

    // Solicita la fecha y fuerza al ciclo a repetirse hasta que sea válida
    public static String solicitarFecha() {
        String fecha;
        while (true) {
            System.out.print(s: " Ingrese fecha (DD/MM/AAAA): ");
            fecha = teclado.next();
            if (validarFecha(fecha)) {
                return fecha;
            }
            System.out.println(x: " [Error] Formato de fecha incorrecto. Use DD/MM/AAAA.");
        }
    }
}
```

2.6.2.2 Código Principal o Fuente

```
public static void main(String[] args) {
    int opcion = 0;

    // Ciclo principal del menú de control de asistencia
    do {
        System.out.println(x: "\n=====");
        System.out.println(x: " SISTEMA CONTROL DE ASISTENCIA ");
        System.out.println(x: "=====");
        System.out.println(x: "1. Registrar Asistencia");
        System.out.println(x: "2. Mostrar Historial y Faltas");
        System.out.println(x: "3. Salir del programa");
        System.out.println(x: "-----");
        System.out.print(s: "Seleccione una opcion (1-3): ");

        // Validación para evitar bucles infinitos por ingreso de letras
        if (teclado.hasNextInt()) {
            opcion = teclado.nextInt();

            switch (opcion) {
                case 1:
                    registrarAsistencia();
                    break;
                case 2:
                    mostrarHistorialYFaltas();
                    break;
                case 3:
                    System.out.println(x: "\nSaliendo del sistema de asistencia... ¡Hasta luego!");
                    break;
                default:
                    System.out.println(x: "[Error] La opcion elegida no existe. Intente con 1, 2 o 3.");
            }
        } else {
            System.out.println(x: "[Error] Entrada no valida. Debe ingresar un numero entero.");
            teclado.next(); // Descarta la entrada incorrecta del buffer
            opcion = 0; // Reinicia la opción para que continúe el ciclo
        }
    } while (opcion != 3);
}
```

2.6.2.3 Ejecución del Programa



```
=====
SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
=====
Seleccione una opcion (1-3): 1

--- NUEVO REGISTRO DE ASISTENCIA ---
Nombre del estudiante: Paulo
Ingrese fecha (DD/MM/AAAA): 25/06/2007
Ingrese estado (A = Asistio, F = Falto): F

>> Asistencia registrada con exito en el archivo.

=====
SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
=====
Seleccione una opcion (1-3): 2

=====
HISTORIAL DE ASISTENCIA
=====
Estudiante      Fecha      Estado
=====
Paulo           25/06/2007  Falto
=====
Total de registros procesados: 1
Total de FALTAS contabilizadas: 1
=====

=====
SISTEMA CONTROL DE ASISTENCIA
=====
1. Registrar Asistencia
2. Mostrar Historial y Faltas
3. Salir del programa
=====
Seleccione una opcion (1-3): █
```

2.7 Ejercicio 7

2.7.1 Código C++

2.7.1.1 Clases y Métodos



```
C Cuenta.h X BancoMain.cpp cajero.txt
C Cuenta.h > ...
1 // Librerías necesarias
2 #include <iostream>
3 #include <fstream>
4
5 using namespace std;
6
7 // Clase Cuenta
8 class Cuenta {
9
10     // Atributos privados
11 private:
12     string usuario;
13     double saldo;
14
15 public:
16
17     // Constructor vacío
18     Cuenta() {
19
20         usuario = "";
21         saldo = 0;
22     }
23
24     // Métodos SET
25     void setUsuario(string u) {
26         usuario = u;
27     }
28
29     void setSaldo(double s) {
30         saldo = s;
31     }
32
33     // Métodos GET
34     string getUsuario() {
35         return usuario;
36     }
37
38     double getSaldo() {
39         return saldo;
40     }
41
42     // Método para depositar dinero
43     void depositar(double monto) {
44
45         saldo += monto;
46     }
47
48     // Método para retirar dinero
49     void retirar(double monto) {
50
51         // Verificar si tiene suficiente saldo
52         if(monto <= saldo) {
53
54             saldo -= monto;
55             cout << "Retiro realizado correctamente" << endl;
56
57         } else {
58
59             cout << "Saldo insuficiente" << endl;
60         }
61     }
62
63     // Método para mostrar saldo
64     void consultarSaldo() {
65
66         cout << "Saldo actual: $" << saldo << endl;
67     }
68 };
```

2.7.1.2 Código Principal o Fuente



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
C Cuenta.h BancoMain.cpp X cajero.txt
C- BancoMain.cpp > main()
1 // Librerías necesarias
2 #include <iostream>
3 #include <fstream>
4 #include "Cuenta.h"
5
6 using namespace std;
7
8 int main() {
9
10     // Crear objeto cuenta
11     Cuenta cuenta;
12
13     // Variables
14     string usuario;
15     double saldoInicial;
16     double monto;
17     int opcion;
18
19     // Archivo para guardar datos
20     ofstream archivo("cajero.txt");
21
22     // Ingreso de datos
23     cout << "Ingrese el nombre del usuario: ";
24     getline(cin, usuario);
25
26     cout << "Ingrese el saldo inicial: ";
27     cin >> saldoInicial;
28
29     // Guardar datos en el objeto
30     cuenta.setUsuario(usuario);
31     cuenta.setSaldo(saldoInicial);
32 }
```

```
33 // Menú principal
34 do {
35
36     cout << "\n===== CAJERO AUTOMATICO =====" << endl;
37     cout << "1. Consultar saldo" << endl;
38     cout << "2. Depositar dinero" << endl;
39     cout << "3. Retirar dinero" << endl;
40     cout << "4. Salir" << endl;
41     cout << "Seleccione una opcion: ";
42     cin >> opcion;
43
44     switch(opcion) {
45
46         case 1:
47
48             // Mostrar saldo
49             cuenta.consultarSaldo();
50             break;
51
52         case 2:
53
54             // Depositar dinero
55             cout << "Ingrese el monto a depositar: ";
56             cin >> monto;
57
58             cuenta.depositar(monto);
59
60             cout << "Deposito realizado correctamente" << endl;
61             break;
62
63         case 3:
64
65             // Retirar dinero
66             cout << "Ingrese el monto a retirar: ";
67             cin >> monto;
68
69             cuenta.retirar(monto);
70             break;
71
72         case 4:
73
74             cout << "Saliendo del sistema..." << endl;
75             break;
76
77         default:
78
79             cout << "Opcion invalida" << endl;
80     }
81 } while(opcion != 4);
82
83 // Guardar datos en archivo
84 archivo << "Usuario: " << cuenta.getUsuario() << endl;
85 archivo << "Saldo final: $" << cuenta.getSaldo() << endl;
86
87 // Cerrar archivo
88 archivo.close();
89
90 return 0;
91
92 }
```

2.7.1.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
C Cuenta.h BancoMain.cpp X cajero.txt
BancoMain.cpp > main()
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\Usuario\OneDrive\Ape05> cd 'c:\Users\Usuario\OneDrive\Ape05\output'
PS C:\Users\Usuario\OneDrive\Ape05\output> & .\BancoMain.exe
Ingrese el nombre del usuario: Karla
Ingrese el saldo inicial: 100

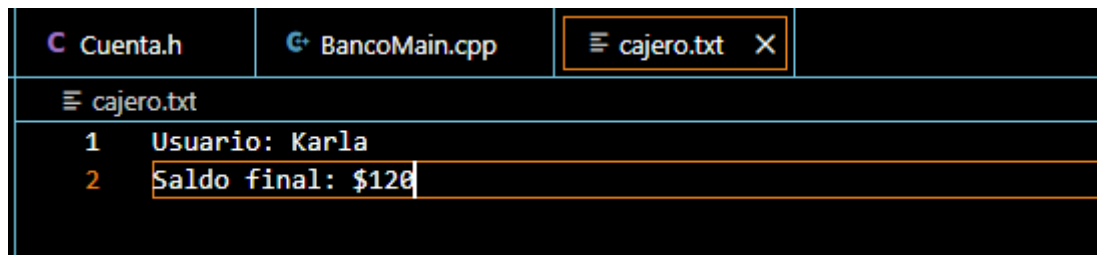
===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 2
Ingrese el monto a depositar: 60
Deposito realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $160

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 3
Ingrese el monto a retirar: 40
Retiro realizado correctamente
```

```
===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $120

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 4
Saliendo del sistema...
PS C:\Users\Usuario\OneDrive\Ape05\output> |
```



2.7.2 Código Java

2.7.2.1 Clases y Métodos

```
J Cuentajava X J BancoMain.java cajerojava.txt
J Cuentajava > Cuenta > usuario
1 // Clase Cuenta
2 public class Cuenta {
3
4     // Atributos privados
5     private String usuario;
6     private double saldo;
7
8     // Constructor vacío
9     public Cuenta() {
10
11         usuario = "";
12         saldo = 0;
13     }
14
15     // Métodos SET
16     public void setUsuario(String usuario) {
17         this.usuario = usuario;
18     }
19
20     public void setSaldo(double saldo) {
21         this.saldo = saldo;
22     }
23
24     // Métodos GET
25     public String getUsuario() {
26         return usuario;
27     }
28
29     public double getSaldo() {
30         return saldo;
31     }
32 }
```

```
33 // Método para depositar dinero
34 public void depositar(double monto) {
35
36     saldo += monto;
37 }
38
39 // Método para retirar dinero
40 public void retirar(double monto) {
41
42     // Verificar saldo disponible
43     if(monto <= saldo) {
44
45         saldo -= monto;
46         System.out.println(x: "Retiro realizado correctamente");
47     } else {
48
49         System.out.println(x: "Saldo insuficiente");
50     }
51 }
52
53 // Método para consultar saldo
54 public void consultarSaldo() {
55
56     System.out.println("Saldo actual: $" + saldo);
57 }
58
59
60
61
62 }
```

2.7.2.2 Código Principal o Fuente



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
J Cuenta.java  J BancoMain.java X  cajerojava.txt

J BancoMain.java > BancoMain > main(String[])

1  // Librerías necesarias
2  import java.util.Scanner;
3  import java.io.FileWriter;
4  import java.io.IOException;
5
6  public class BancoMain{
7
8      Run | Debug
      public static void main(String[] args) {
9
10         // Objeto Scanner
11         Scanner entrada = new Scanner(System.in);
12
13         // Crear objeto cuenta
14         Cuenta cuenta = new Cuenta();
15
16         // Variables
17         String usuario;
18         double saldoInicial;
19         double monto;
20         int opcion;
21
22         try {
23
24             // Archivo para guardar datos
25             FileWriter archivo = new FileWriter(fileName: "cajerojava.txt");
26
27             // Ingreso de datos
28             System.out.print(s: "Ingrese el nombre del usuario: ");
29             usuario = entrada.nextLine();
30
31             System.out.print(s: "Ingrese el saldo inicial: ");

6  public class BancoMain{
8      public static void main(String[] args) {
32         saldoInicial = entrada.nextDouble();
33
34         // Guardar datos en el objeto
35         cuenta.setUsuario(usuario);
36         cuenta.setSaldo(saldoInicial);
37
38         // Menú principal
39         do {
40
41             System.out.println(x: "\n===== CAJERO AUTOMATICO =====");
42             System.out.println(x: "1. Consultar saldo");
43             System.out.println(x: "2. Depositar dinero");
44             System.out.println(x: "3. Retirar dinero");
45             System.out.println(x: "4. Salir");
46             System.out.print(s: "Seleccione una opcion: ");
47             opcion = entrada.nextInt();
48
49             switch(opcion) {
50
51                 case 1:
52
53                     // Consultar saldo
54                     cuenta.consultarSaldo();
55                     break;
56
57                 case 2:
58
59                     // Depositar dinero
60                     System.out.print(s: "Ingrese el monto a depositar: ");
61                     monto = entrada.nextDouble();
```



```
6 public class BancoMain{
8     public static void main(String[] args) {
11         // ...
12
13         cuenta.depositar(monto);
14
15         System.out.println(x: "Deposito realizado correctamente");
16         break;
17
18         case 3:
19
20             // Retirar dinero
21             System.out.print(s: "Ingrese el monto a retirar: ");
22             monto = entrada.nextDouble();
23
24             cuenta.retirar(monto);
25             break;
26
27         case 4:
28
29             System.out.println(x: "Saliendo del sistema...");
30             break;
31
32         default:
33
34             System.out.println(x: "Opcion invalida");
35
36     }
37
38     } while(opcion != 4);
39
40     // Guardar datos en archivo
41     archivo.write("Usuario: " + cuenta.getUsuario() + "\n");
42
43     // ...
44
45     archivo.write("Saldo final: $" + cuenta.getSaldo() + "\n");
46
47     // Cerrar archivo
48     archivo.close();
49
50     } catch(IOException e) {
51
52         System.out.println(x: "Error al crear el archivo");
53     }
54
55     // Cerrar Scanner
56     entrada.close();
57
58 }
59 }
```

2.7.2.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
J Cuenta.java X J BancoMain.java cajerojava.txt
J Cuenta.java > Cuenta > usuario
2 public class Cuenta {
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\Usuario\OneDrive\Ape05> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDe
ExceptionMessages' '-cp' 'C:\Users\Usuario\AppData\Roaming\Code\User\workspaceStorage\31acb19243734
52fc7c06bf91\redhat.java\jdt_ws\Ape05_d35a3370\bin' 'BancoMain'
Ingrese el nombre del usuario: Juan
Ingrese el saldo inicial: 30

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $30.0

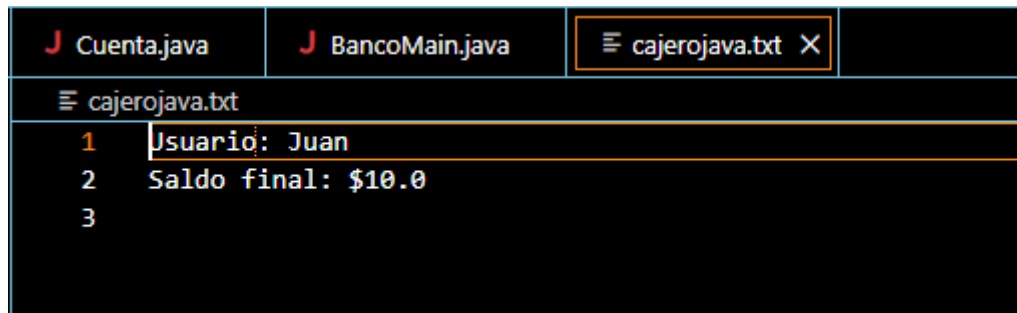
===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 2
Ingrese el monto a depositar: 30
Deposito realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $60.0

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 3
Ingrese el monto a retirar: 50
Retiro realizado correctamente

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 1
Saldo actual: $10.0

===== CAJERO AUTOMATICO =====
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Seleccione una opcion: 4
Saliendo del sistema...
PS C:\Users\Usuario\OneDrive\Ape05>
```



2.8 Ejercicio 8

2.8.1 Código C++

2.8.1.1 Clases y Métodos

```
class Biblioteca {
private:
    string nombreArchivo = "biblioteca.txt";

public:
    // Método para registrar un libro
    void registrarLibro(Libro libro) {
        ofstream archivo(nombreArchivo, ios::app); // ios::app para agregar al final
        if (archivo.is_open()) {
            archivo << libro.toString() << "\n";
            archivo.close();
            cout << "¡Libro registrado con éxito en el archivo!\n";
        } else {
            cout << "Error al abrir el archivo para guardar.\n";
        }
    }

    // Método para buscar un libro por código o título
    void buscarLibro(string criterio) {
        ifstream archivo(nombreArchivo);
        string linea;
        bool encontrado = false;

        if (archivo.is_open()) {
            while (getline(archivo, linea)) {
                stringstream ss(linea);
                string cod, tit, aut, est;
```

2.8.1.2 Código Principal o Fuente



```
if (opcion == 1) {
    string cod, tit, aut, est;
    int estOpcion;
    cout << "Código: "; getline(cin, cod);
    cout << "Título: "; getline(cin, tit);
    cout << "Autor: "; getline(cin, aut);
    cout << "Estado (1. Terminado / 2. En proceso): "; cin >> estOpcion;
    est = (estOpcion == 1) ? "Terminado" : "En proceso";

    Libro nuevoLibro(cod, tit, aut, est);
    miBiblioteca.registrarLibro(nuevoLibro);
} else if (opcion == 2) {
    string criterio;
    cout << "Ingrese código o título a buscar: ";
    getline(cin, criterio);
    miBiblioteca.buscarLibro(criterio);
} else if (opcion == 3) {
    miBiblioteca.mostrarPorEstado();
}
} while (opcion != 4);

return 0;
}
```

2.8.1.3 Ejecución del Programa

```
=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===

--- Datos del Estudiante 1 ---
Ingrese Cedula: 1850342724
Ingrese Nombre: Diego
Ingrese Apellido: Balarezo
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9.5

--- Datos del Estudiante 2 ---
Ingrese Cedula: 1805695826
Ingrese Nombre: Mateo
Ingrese Apellido: Vargas
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 7
Ingrese la Nota 3 (0 - 10): 10

--- Datos del Estudiante 3 ---
Ingrese Cedula: 1800083416
Ingrese Nombre: Patricio
Ingrese Apellido: Ramirez
Ingrese la Nota 1 (0 - 10): 9
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 6

--- Datos del Estudiante 4 ---
Ingrese Cedula: 1856478934
Ingrese Nombre: Ramiro
Ingrese Apellido: Jacome
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 6
Ingrese la Nota 3 (0 - 10): 8
```



2.8.2 Código Java

2.8.2.1 Clases y Métodos

```
class Biblioteca {  
    private List<Libro> libros = new ArrayList<>();  
    private final String ARCHIVO = "libros.txt";  
  
    // Constructor: carga libros existentes  
    public Biblioteca() { cargarDesdeArchivo(); }  
  
    // — Cargar todos los libros desde el archivo —  
    private void cargarDesdeArchivo() {  
        libros.clear();  
        File f = new File(ARCHIVO);  
        if (!f.exists()) return;  
  
        try (BufferedReader br = new BufferedReader(new FileReader(f))) {  
            String linea;  
            while ((linea = br.readLine()) != null) {  
                if (!linea.isBlank())  
                    libros.add(Libro.desdeTexto(linea));  
            }  
        } catch (IOException e) {  
            System.out.println("[!] Error al leer archivo: " + e.getMessage());  
        }  
    }  
  
    // — Guardar todos los libros en el archivo —  
    private void guardarEnArchivo() {  
        try (PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO))) {
```

2.8.2.2 Código Principal o Fuente

```
        opcion = Integer.parseInt(sc.nextLine());  
  
        switch (opcion) {  
            case 1 -> bib.registrarLibro(sc);  
            case 2 -> bib.buscarLibro(sc);  
            case 3 -> bib.mostrarDisponibles();  
            case 4 -> bib.mostrarPrestados();  
            case 5 -> bib.mostrarTodos();  
            case 0 -> System.out.println(x: "Hasta luego!");  
            default -> System.out.println(x: "[!] Opcion invalida.");  
        }  
    } while (opcion != 0);  
  
    sc.close();  
}
```

2.8.2.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



=== REGISTRO DE ESTUDIANTES - ALGORITMOS ===

```
--- Datos del Estudiante 1 ---
Ingreso Cedula: 1850342724
Ingreso Nombre: Diego
Ingreso Apellido: Balarezo
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9.5

--- Datos del Estudiante 2 ---
Ingreso Cedula: 1805695826
Ingreso Nombre: Mateo
Ingreso Apellido: Vargas
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 7
Ingrese la Nota 3 (0 - 10): 10

--- Datos del Estudiante 3 ---
Ingreso Cedula: 1800083416
Ingreso Nombre: Patricio
Ingreso Apellido: Ramirez
Ingrese la Nota 1 (0 - 10): 9
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 6

--- Datos del Estudiante 4 ---
Ingreso Cedula: 1856478934
Ingreso Nombre: Ramiro
Ingreso Apellido: Jacome
Ingrese la Nota 1 (0 - 10): 7
Ingrese la Nota 2 (0 - 10): 6
Ingrese la Nota 3 (0 - 10): 8
```

```
--- Datos del Estudiante 5 ---
Ingreso Cedula: 1857362659
Ingreso Nombre: Sebastian
Ingreso Apellido: Lopez
Ingrese la Nota 1 (0 - 10): 8
Ingrese la Nota 2 (0 - 10): 9
Ingrese la Nota 3 (0 - 10): 9
```

=====

LISTADO DE ESTUDIANTES

=====

Cedula: 1850342724	Estudiante: Balarezo Diego	Notas: [7, 9, 9.5]	Promedio: 8.50	Estado: Aprobado
Cedula: 1805695826	Estudiante: Vargas Mateo	Notas: [8.00, 7.00, 10.00]	Promedio: 8.33	Estado: Aprobado
Cedula: 1800083416	Estudiante: Ramirez Patricio	Notas: [9.00, 9.00, 6.00]	Promedio: 8.00	Estado: Aprobado
Cedula: 1856478934	Estudiante: Jacome Ramiro	Notas: [7.00, 6.00, 8.00]	Promedio: 7.00	Estado: Aprobado
Cedula: 1857362659	Estudiante: Lopez Sebastian	Notas: [8.00, 9.00, 9.00]	Promedio: 8.67	Estado: Aprobado

=====

RESUMEN DE RENDIMIENTO

Total Estudiantes Aprobados: 5
Total Estudiantes Reprobados: 0

=====

PS C:\Users\disba\Documents\Diegui\UTA Primer Semestre\VSCodeProjects> █



2.9 Ejercicio 9

2.9.1 Código C++

2.9.1.1 Clases y Métodos

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include <sstream>

using namespace std;

// =====
// CLASE VENTA (Aplicación de P00)
// =====

class Venta {
private:
    // Atributos encapsulados
    string producto;
    int cantidad;
    double precio;
    double total;

public:
    // Constructor por defecto
    Venta() {
        producto = "";
        cantidad = 0;
        precio = 0.0;
        total = 0.0;
    }

    // Métodos Setters y Getters (Métodos de acceso)
    void setProducto(string prod) { producto = prod; }
    void setCantidad(int cant) { cantidad = cant; }
    void setPrecio(double prec) { precio = prec; }

    string getProducto() { return producto; }
    int getCantidad() { return cantidad; }
    double getPrecio() { return precio; }
    double getTotal() { return total; }

    // Método para calcular el total automáticamente
    void calcularTotal() {
        total = cantidad * precio;
    }
}
```



2.9.1.2 Código Principal o Fuente

```
int main() {
    do {
        cout << "\n===== \n";
        cout << "        SISTEMA DE VENTAS        \n";
        cout << "===== \n";
        cout << "1. Registrar Nueva Venta (Facturar)\n";
        cout << "2. Mostrar Historial de Ventas\n";
        cout << "3. Salir del programa\n";
        cout << "===== \n";
        cout << "Seleccione una opcion (1-3): ";

        // Validación para evitar bucles infinitos con caracteres inválidos
        if (!cin >> opcion) {
            cout << "[Error] Entrada invalida. Intente de nuevo.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (opcion) {
            case 1: {
                Venta nuevaVenta; // Instanciación del objeto
                string prod;
                int cant;
                double precio;

                cout << "\n--- INGRESO DE DATOS DE LA VENTA ---\n";
                cout << "Nombre del producto: ";
                cin.ignore(); // Limpia el buffer
                getline(cin, prod);
                nuevaVenta.setProducto(prod);

                // Validación de cantidad
                do {
                    cout << "Cantidad: ";
                    if (cin >> cant && cant > 0) break;
                    cout << "[Error] Cantidad debe ser un entero mayor a 0.\n";
                    cin.clear(); cin.ignore(10000, '\n');
                } while (true);
                nuevaVenta.setCantidad(cant);

                // Validación de precio
            }
        }
    } while (true);
}
```

2.9.1.3 Ejecución del Programa

```
=====
        SISTEMA DE VENTAS
=====
1. Registrar Nueva Venta (Facturar)
2. Mostrar Historial de Ventas
3. Salir del programa
=====
Seleccione una opcion (1-3): 1

--- INGRESO DE DATOS DE LA VENTA ---
Nombre del producto: POLLO
Cantidad: 12
Precio unitario: 10

=====
        FACTURA DE VENTA
=====
Producto:    POLLO
Cantidad:    12
Precio Unit: $10.00
=====
TOTAL:       $120.00
=====

=====
        SISTEMA DE VENTAS
=====
1. Registrar Nueva Venta (Facturar)
2. Mostrar Historial de Ventas
3. Salir del programa
=====
Seleccione una opcion (1-3):
```

2.9.2 Código Java

2.9.2.1 Clases y Métodos



```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

// =====
// CLASE VENTA (Aplicación estricta de POO)
// =====
class Venta {
    // Atributos privados (Encapsulamiento)
    private String producto;
    private int cantidad;
    private double precio;
    private double total;

    // Constructor por defecto
    public Venta() {
        this.producto = "";
        this.cantidad = 0;
        this.precio = 0.0;
        this.total = 0.0;
    }

    // Métodos Setters y Getters
    public void setProducto(String producto) { this.producto = producto; }
    public void setCantidad(int cantidad) { this.cantidad = cantidad; }
    public void setPrecio(double precio) { this.precio = precio; }

    public String getProducto() { return producto; }
    public int getCantidad() { return cantidad; }
    public double getPrecio() { return precio; }
    public double getTotal() { return total; }

    // Método para calcular el total de forma autónoma
    public void calcularTotal() {
        this.total = this.cantidad * this.precio;
    }

    // REQUERIMIENTO: Guardar el registro actual en el archivo de texto
}
```

2.9.2.2 Código Principal o Fuente

2.9.2.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
=====
SISTEMA DE VENTAS
=====
1. Registrar Nueva Venta (Facturar)
2. Mostrar Historial de Ventas
3. Salir del programa
=====
Seleccione una opcion (1-3): 2

[Aviso] No existen ventas registradas en el historial (Archivo no encontrado).

=====
SISTEMA DE VENTAS
=====
1. Registrar Nueva Venta (Facturar)
2. Mostrar Historial de Ventas
3. Salir del programa
=====
Seleccione una opcion (1-3): 1

--- INGRESO DE DATOS DE LA VENTA ---
Nombre del producto: PAPAS
Cantidad: 45
Precio unitario: 7

=====
FACTURA DE VENTA
=====
Producto: PAPAS
Cantidad: 45
Precio Unit: $7.00
=====
TOTAL: $315.00
=====

=====
SISTEMA DE VENTAS
=====
1. Registrar Nueva Venta (Facturar)
2. Mostrar Historial de Ventas
3. Salir del programa
=====
Seleccione una opcion (1-3): █
```

2.10 Ejercicio 10

2.10.1 Código C++

2.10.1.1 Clases y Métodos



```
class Contacto {
public:
    string nombre, telefono, correo;

    Contacto(string n, string t, string c) {
        nombre = n;
        telefono = t;
        correo = c;
    }

    string toString() {
        return nombre + "," + telefono + "," + correo;
    }
};
```

```
void agregar() {
    ofstream file(archivo, ios::app);
    string n, t, c;

    cout << "Nombre: ";
    getline(cin, n);
    cout << "Telefono: ";
    getline(cin, t);
    cout << "Correo: ";
    getline(cin, c);

    Contacto con(n, t, c);
    file << con.toString() << endl;

    file.close();
}
```



```
void mostrar() {  
    ifstream file(archivo);  
    string linea;  
  
    while (getline(file, linea)) {  
        cout << linea << endl;  
    }  
  
    file.close();  
}
```



```
void buscar() {  
    ifstream file(archivo);  
    string linea, nombre;  
    bool encontrado = false;  
  
    cout << "Nombre a buscar: ";  
    getline(cin, nombre);  
  
    while (getline(file, linea)) {  
        if (linea.find(nombre) != string::npos) {  
            cout << "Encontrado: " << linea << endl;  
            encontrado = true;  
        }  
    }  
  
    if (!encontrado)  
        cout << "No encontrado\n";  
  
    file.close();  
}
```




```
void eliminar() {
    ifstream file(archivo);
    ofstream temp("temp.txt");
    string linea, nombre;

    cout << "Nombre a eliminar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) == string::npos) {
            temp << linea << endl;
        }
    }

    file.close();
    temp.close();

    remove("agenda.txt");
    rename("temp.txt", "agenda.txt");

    cout << "Contacto eliminado\n";
}
```

2.10.1.2 Código Principal o Fuente



```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Contacto {
public:
    string nombre, telefono, correo;

    Contacto(string n, string t, string c) {
        nombre = n;
        telefono = t;
        correo = c;
    }

    string toString() {
        return nombre + "," + telefono + "," + correo;
    }
};

string archivo = "agenda.txt";

void agregar() {
    ofstream file(archivo, ios::app);
    string n, t, c;
```



```
    cout << "Nombre: ";
    getline(cin, n);
    cout << "Telefono: ";
    getline(cin, t);
    cout << "Correo: ";
    getline(cin, c);

    Contacto con(n, t, c);
    file << con.toString() << endl;

    file.close();
}

void mostrar() {
    ifstream file(archivo);
    string linea;

    while (getline(file, linea)) {
        cout << linea << endl;
    }

    file.close();
}
```



```
void buscar() {
    ifstream file(archivo);
    string linea, nombre;
    bool encontrado = false;

    cout << "Nombre a buscar: ";
    getline(cin, nombre);

    while (getline(file, linea)) {
        if (linea.find(nombre) != string::npos) {
            cout << "Encontrado: " << linea << endl;
            encontrado = true;
        }
    }

    if (!encontrado)
        cout << "No encontrado\n";

    file.close();
}

void eliminar() {
    ifstream file(archivo);
    ofstream temp("temp.txt");
    string linea, nombre;
```



```
cout << "Nombre a eliminar: ";
getline(cin, nombre);

while (getline(file, linea)) {
    if (linea.find(nombre) == string::npos) {
        temp << linea << endl;
    }
}

file.close();
temp.close();

remove("agenda.txt");
rename("temp.txt", "agenda.txt");

cout << "Contacto eliminado\n";
}

int main() {
    int op;

    do {
        cout << "\n1. Agregar\n2. Mostrar\n3. Buscar\n4. Eliminar\n5. Salir\n";
        cout << "Opcion: ";
        cin >> op;

        cout << "Nombre a eliminar: ";
        getline(cin, nombre);

        while (getline(file, linea)) {
            if (linea.find(nombre) == string::npos) {
                temp << linea << endl;
            }
        }

        file.close();
        temp.close();

        remove("agenda.txt");
        rename("temp.txt", "agenda.txt");

        cout << "Contacto eliminado\n";
    }

    int main() {
        int op;

        do {
            cout << "\n1. Agregar\n2. Mostrar\n3. Buscar\n4. Eliminar\n5. Salir\n";
            cout << "Opcion: ";
            cin >> op;
```

2.10.1.3 Ejecución del Programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 1
Nombre: Nancy
Telefono: 0997745824
Correo: nancyp@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 2
Michael,0979350478,mruiz3405@gmail.com
Michael,0979350748,msanchez3405@gmail.com
Nancy,0997745824,nancyp@gmail.com
```

```
3. Buscar
4. Eliminar
5. Salir
Opcion: 3
Nombre a buscar: Nancy
Encontrado: Nancy,0997745824,nancyp@gmail.com
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 4
Nombre a eliminar: Nancy
Contacto eliminado
```

```
1. Agregar
2. Mostrar
3. Buscar
4. Eliminar
5. Salir
Opcion: 5
PS C:\Users\HP\Documents\ALGORITMOS\Agenda>
```